

CyberGuard Analytics - Network Intrusion Detection System

CRISP-DM Design Document

CyberGuard Analytics Data Science Team

2025-11-30

Table of contents

Executive Summary	8
Data Architecture and Pipeline Design	8
Overview	8
Data Sources	8
Primary Data Source: NETWORK_TRAFFIC_HISTORY Database	8
Data Schema	9
Data Pipeline Architecture	9
Ingestion Layer	9
Processing Layer	9
Storage Architecture	9
Integration Points	10
Data Flow Diagram	10
Model Architecture Specification	10
Overview	10
Model Selection Rationale	10
Model Architecture	11
1. APT (Advanced Persistent Threat) Detection Model	11
2. Insider Threat Detection (ITD) Model	11
3. Phishing Detection Model	11
4. Zero-day Exploit Detection Model	12
Feature Engineering Approach	12
Categorical Encoding	12
Derived Features	12

Feature Selection	12
Model Training Process	13
Data Preparation	13
Training Methodology	13
Model Persistence	13
Model Inference Architecture	13
Infrastructure and Technology Stack	14
Cloud Infrastructure (AWS)	14
Compute Services	14
Data Services	14
Monitoring and Logging	14
Development Tools	15
Data Science Environment	15
Database Tools	15
Environment Management	15
CI/CD Pipeline	15
Security Infrastructure	16
Data Quality and Validation Framework	16
Data Quality Dimensions	16
Completeness	16
Accuracy	16
Consistency	16
Timeliness	17
Data Validation Procedures	17
Pre-Ingestion Validation	17
Post-Processing Validation	17
Data Quality Monitoring	17
Automated Monitoring	17
Manual Quality Reviews	17
Data Cleansing Procedures	18
Automated Cleansing	18
Manual Intervention	18
Security and Privacy Controls	18
Data Protection Measures	18
Encryption	18
Access Controls	18
Compliance Requirements	19
Data Privacy Regulations	19
Industry Standards	19

Privacy Safeguards	19
Data Anonymization	19
Data Usage Agreements	19
Audit and Logging	19
Ethical Considerations	20
Bias Mitigation	20
Transparency	20
Responsible AI	20
Deployment Strategy and Environment Setup	20
Environment Architecture	20
Development Environment	20
Staging Environment	20
Production Environment	21
Deployment Procedures	21
Model Deployment Workflow	21
Containerization Strategy	21
Infrastructure as Code	21
Rollback Procedures	22
Automated Rollback Triggers	22
Rollback Process	22
Rollback Capabilities	22
Infrastructure Provisioning	22
Automated Provisioning	22
Manual Provisioning	22
Monitoring and Maintenance Plan	23
Performance Tracking	23
Model Performance Metrics	23
System Performance Metrics	23
Monitoring Dashboards	23
Model Drift Detection	23
Data Drift Monitoring	23
Detection Mechanisms	24
Retraining Schedules	24
Scheduled Retraining	24
Retraining Process	24
Model Versioning	24
Ongoing Maintenance Procedures	24
Regular Maintenance Tasks	24
Proactive Maintenance	25
Maintenance Windows	25

Testing and Validation Approach	25
Unit Testing	25
Model Component Testing	25
Code Quality	25
Integration Testing	26
Pipeline Integration Tests	26
System Integration Tests	26
Model Validation Methods	26
Cross-Validation	26
Holdout Validation	26
Temporal Validation	26
Performance Validation	27
Acceptance Criteria	27
Model Performance Criteria	27
Operational Criteria	27
Business Criteria	27
Test Data Management	27
Test Data Strategy	27
Test Environment	28
 API and Interface Specifications	 28
Model Inference API	28
REST API Endpoints	28
Batch Prediction API	29
SIEM/SOAR Integration	29
Splunk Integration	29
QRadar Integration	29
SOAR Platform Integration	29
Client Portal Interface	30
Dashboard API	30
Alert Management API	31
Data Formats	31
Input Data Format	31
Output Data Format	31
Communication Protocols	31
HTTP/HTTPS	31
Message Queue (Optional)	31
WebSocket (Real-time)	31
 Scalability and Performance Considerations	 32
Expected Load	32
Current Estimates	32
Growth Projections	32

Scaling Strategies	32
Horizontal Scaling	32
Vertical Scaling	32
Caching Strategies	32
Performance Benchmarks	33
Latency Targets	33
Throughput Targets	33
Resource Optimization	33
Cost Optimization	33
Performance Tuning	33
Capacity Planning	33
Monitoring and Alerts	33
Planning Process	34
Risk Mitigation and Contingency Plans	34
Technical Risks	34
Model Performance Degradation	34
Data Quality Issues	34
Infrastructure Failures	34
Integration Failures	35
Data and Security Risks	35
Data Breach	35
Compliance Violations	35
Business Risks	36
Project Delays	36
Budget Overruns	36
Failure Scenarios	36
Complete System Failure	36
Model Failure	36
Data Pipeline Failure	37
Risk Monitoring	37
Risk Register	37
Risk Reporting	37
Pilot Phase Implementation Plan	37
Pilot Scope	37
Target User Groups	37
Limited Feature Sets	37
Data Subsets	38
Pilot Timeline	38
Phase 1: Preparation (2 weeks)	38
Phase 2: Soft Launch (1 week)	38
Phase 3: Active Pilot (4 weeks)	38

Phase 4: Evaluation (1 week)	38
Success Metrics	39
Technical Metrics	39
Business Metrics	39
Rollback Procedures	39
Automatic Rollback Triggers	39
Manual Rollback Process	39
Rollback Criteria	39
Criteria for Advancing to Production	40
Technical Criteria	40
Business Criteria	40
Go/No-Go Decision	40
Pilot Phase Business Value Estimation	40
Baseline Measurements	40
Current State Metrics	40
Projected Improvements	41
Detection Performance Improvements	41
Operational Efficiency Gains	41
Business Impact	41
Risk-Adjusted Value Calculations	41
Revenue Impact	41
Cost Savings	42
Pilot Phase ROI	42
Methodology for Measuring Outcomes	42
Measurement Framework	42
Key Performance Indicators	42
Comparison Methodology	43
Reporting	43
Production Phase Implementation Plan	43
Phased Rollout Strategy	43
Phase 1: Foundation (Weeks 1-4)	43
Phase 2: Limited Production (Weeks 5-8)	43
Phase 3: Full Production (Weeks 9-12)	44
User Onboarding Procedures	44
Client Onboarding Process	44
Training Delivery	44
Change Management Protocols	44
Communication Strategy	44
Change Control Process	45
Rollback Procedures	45

System Cutover Activities	45
Cutover Planning	45
Cutover Checklist	45
Production Infrastructure Scaling	46
Infrastructure Requirements	46
Scaling Triggers	46
Data Migration Procedures	46
Migration Strategy	46
Migration Validation	46
User Access Provisioning	47
Access Management	47
Access Types	47
Training Delivery Timelines	47
Training Schedule	47
Communication Strategies	47
Internal Communication	47
Client Communication	48
Organization-Wide Adoption	48
Post-Production Monitoring Plan	48
Automated Monitoring Dashboards	48
Real-Time Dashboards	48
Custom Dashboards	48
Alert Systems	49
Alert Configuration	49
Alert Thresholds	49
Performance Tracking Mechanisms	49
Model Performance Monitoring	49
System Performance Monitoring	49
Model Performance Degradation Detection	50
Detection Mechanisms	50
Degradation Response	50
Data Quality Monitoring	50
Quality Metrics	50
Quality Alerts	50
Business Impact Measurement	51
Business Metrics	51
Impact Reporting	51
Ongoing Maintenance Procedures	51
Regular Maintenance	51
Proactive Maintenance	51
Maintenance Schedule	52
Scheduled Maintenance Windows	52

Emergency Maintenance	52
Appendices	52
Appendix A: Glossary	52
Appendix B: References	52
Appendix C: Document Version History	53

Executive Summary

This document outlines the comprehensive design for CyberGuard Analytics' specialized network intrusion detection system. The project focuses on developing four specialized anomaly detection models targeting Advanced Persistent Threats (APT), Insider Threat Detection (ITD), Phishing, and Zero-day exploits. The system leverages XGBoost machine learning models deployed on AWS infrastructure to provide real-time threat detection capabilities.

Project Objectives: - Reduce Mean Time to Detect (MTTD) by 50% per attack type - Maintain False Positive Rate below 2% - Achieve 18-22% ROI within 3 years - Integrate seamlessly with existing SIEM/SOAR platforms

Data Architecture and Pipeline Design

Overview

The data architecture is built on AWS cloud infrastructure with PostgreSQL as the primary data store. The system processes network flow data from multiple enterprise clients in real-time and batch modes.

Data Sources

Primary Data Source: NETWORK_TRAFFIC_HISTORY Database

Location: AWS RDS PostgreSQL instance

Schema: network_traffic_history table containing:

- **Flow Records:** Timestamp, source/destination IPs, ports, protocols, packet/byte counts, TCP flags, service types
- **Client Metadata:** Network configurations and security policies
- **Historical Attack Labels:** Confirmed cyberattacks with timestamps, attack types, affected systems

Data Volume: ~5 million records per dataset, with continuous ingestion

Data Schema

Key columns include: - `timestamp` - Flow timestamp - `source_ip`, `dest_ip` - Network addresses - `source_port`, `dest_port` - Port numbers - `protocol` - Network protocol (TCP, UDP, ICMP) - `duration`, `packets`, `bytes` - Flow metrics - `bytes_per_packet`, `packets_per_second` - Derived metrics - `tcp_flags` - TCP flag combinations - `service` - Detected service type - `attack_state` - Attack classification label - `severity_score` - Threat severity rating - Temporal features: `is_weekend`, `hour_of_day`, `day_of_week` - Behavioral features: `bytes_ratio`, `packet_size_variance`, `connection_frequency`, `unique_ports_per_source`

Data Pipeline Architecture

Ingestion Layer

Real-time Stream Processing: - **AWS Kinesis Data Streams:** Ingests network flow data from client environments - **Kinesis Data Firehose:** Buffers and batches data for efficient database writes - **Lambda Functions:** Pre-process data, validate schemas, apply initial transformations

Batch Processing: - Scheduled ETL jobs extract historical data from client SIEM systems - Data validation and quality checks before database insertion - Incremental updates to minimize processing overhead

Processing Layer

ETL Pipeline (Jupyter Notebooks): - `data_preparation.ipynb` - Creates attack-specific datasets - `etl.ipynb` - Orchestrates data transformation workflows - `data_understanding.ipynb` - Exploratory data analysis

Data Transformation Steps: 1. Extract raw network flow data from PostgreSQL 2. Filter and segment by attack type (APT, ITD, Phishing, Zero-day) 3. Feature engineering and encoding 4. Data quality validation 5. Write processed datasets back to database

Storage Architecture

PostgreSQL Database Structure: - Raw data: `network_traffic_history` (source of truth) - Processed datasets: `network_traffic_history_appt`, `network_traffic_history_itd`, `network_traffic_history_phishing`, `network_traffic_history_zeroday` - Model meta-data: Model versions, performance metrics, training parameters - Feature stores: Pre-computed features for model inference

AWS S3: - Model artifacts (.pkl files) - Training datasets (backup and versioning) - Log files and audit trails

Integration Points

Client Systems: - SIEM platforms (Splunk, QRadar) - Data ingestion via APIs - IDS/Firewalls - Network flow export - SOAR platforms - Alert integration

Internal Systems: - AWS SageMaker - Model training and deployment - CloudWatch - Monitoring and logging - Client Portal - Dashboard and reporting

Data Flow Diagram

```
Client Networks → Kinesis Streams → Lambda (Preprocessing) → PostgreSQL
                                                                ↓
PostgreSQL → ETL Pipeline → Feature Engineering → Model Training
                                                                ↓
Model Inference ← Feature Store ← Real-time Features ← Kinesis
                                                                ↓
Alerts → SIEM/SOAR → SOC Teams
```

Model Architecture Specification

Overview

Four specialized XGBoost classification models, each optimized for a specific attack type. All models follow a consistent architecture with attack-specific feature engineering.

Model Selection Rationale

XGBoost Selection: - Handles mixed data types (categorical, numerical, temporal) - Robust to missing values and outliers - Provides feature importance for explainability - Efficient training and inference - Proven performance in cybersecurity applications

Model Architecture

1. APT (Advanced Persistent Threat) Detection Model

Model Type: Binary Classification (XGBoost)

Target Variable: `is_apt` (boolean)

Training Data: `network_traffic_history_apt`

Key Features: - Long-term behavioral patterns (connection frequency over weeks) - Multi-vector coordination indicators - Gradual pattern evolution metrics - Temporal features (hour, day, weekend patterns) - Protocol and service combinations

Model Parameters (Typical): - Objective: `binary:logistic` - Evaluation metric: `auc` - Tree method: `hist` (for efficiency) - Learning rate: 0.01-0.1 (tuned) - Max depth: 6-10 (tuned) - Subsampling: 0.8-0.9

2. Insider Threat Detection (ITD) Model

Model Type: Binary Classification (XGBoost)

Target Variable: `is_insider_threat` (boolean)

Training Data: `network_traffic_history_itd`

Key Features: - Off-hours access patterns - Unusual data access volumes - Privilege escalation indicators - User behavior baselines - Internal network communication patterns

Model Parameters: Similar to APT model with emphasis on temporal and behavioral features

3. Phishing Detection Model

Model Type: Binary Classification (XGBoost)

Target Variable: `is_phishing` (boolean)

Training Data: `network_traffic_history_phishing`

Key Features: - Domain reputation indicators - HTTP request patterns to suspicious domains - Credential submission patterns - Email-related connection timing - Payload pattern analysis

Model Parameters: Optimized for domain and timing features

4. Zero-day Exploit Detection Model

Model Type: Binary Classification (XGBoost)

Target Variable: `is_zeroday` (boolean)

Training Data: `network_traffic_history_zeroday`

Key Features: - Novel traffic patterns - Unexpected service responses - Protocol anomalies - Payload anomalies - Unprecedented behavior indicators

Model Parameters: Focus on anomaly detection features

Feature Engineering Approach

Categorical Encoding

Label Encoding Applied To: - `protocol` (TCP, UDP, ICMP) - `tcp_flags` (SYN, ACK, FIN, RST, etc.) - `service` (HTTP, SMTP, DNS, SSH, etc.) - `is_weekend` (boolean) - `source_ip`, `dest_ip` (IP addresses) - `source_port`, `dest_port` (port numbers) - `timestamp` (temporal encoding)

Derived Features

Temporal Features: - `hour_of_day` - Hour of day (0-23) - `day_of_week` - Day of week (0-6) - `is_weekend` - Weekend indicator

Statistical Features: - `bytes_ratio` - Ratio of bytes in/out - `packet_size_variance` - Variance in packet sizes - `bytes_per_packet` - Average bytes per packet - `packets_per_second` - Connection rate

Behavioral Features: - `connection_frequency` - Frequency of connections from source - `unique_ports_per_source` - Port diversity indicator

Feature Selection

- Remove highly correlated features
- Drop non-informative columns (e.g., `attack_state` in training)
- Retain features with high importance scores from XGBoost

Model Training Process

Data Preparation

1. **Data Extraction:** Query PostgreSQL for attack-specific datasets
2. **Feature/Target Separation:** Split into X (features) and y (target)
3. **Categorical Encoding:** Apply LabelEncoder to categorical variables
4. **Handling Imbalance:** SMOTE oversampling for minority class (if needed)
5. **Train/Test Split:** Stratified split (80/20) maintaining class distribution

Training Methodology

Cross-Validation: - Stratified K-Fold (5 folds) for robust evaluation - Prevents overfitting and provides performance estimates

Hyperparameter Tuning: - Grid search or Bayesian optimization - Focus on: learning_rate, max_depth, n_estimators, subsample, colsample_bytree

Model Evaluation Metrics: - Accuracy, Precision, Recall, F1-Score - ROC-AUC Score - Matthews Correlation Coefficient (MCC) - Cohen's Kappa - Confusion Matrix

Model Persistence

Storage Format: Pickle (.pkl files) - apt_xgboost_model.pkl - itd_xgboost_model.pkl - phishing_xgboost_model.pkl - zeroday_xgboost_model.pkl

Version Control: - Model versioning with timestamps - Performance metrics stored with each version - Metadata: training date, data snapshot, hyperparameters

Model Inference Architecture

Real-time Scoring: - Lambda functions load models from S3 - Feature extraction from incoming network flows - Prediction generation with confidence scores - Alert generation for high-confidence threats

Batch Scoring: - Scheduled jobs process historical data - Retrospective threat detection - Model performance validation

Infrastructure and Technology Stack

Cloud Infrastructure (AWS)

Compute Services

AWS SageMaker: - Model training and hyperparameter tuning - Model registry and versioning
- Endpoint deployment for inference - Built-in monitoring and logging

AWS Lambda: - Real-time data preprocessing - Model inference functions - Alert generation and routing - Event-driven processing

AWS ECS/EKS (Container Orchestration): - Docker containerization for model serving
- Kubernetes for scalable deployment - Consistent environments across dev/staging/prod

Data Services

AWS RDS PostgreSQL: - Primary data store for network flow data - Automated backups and point-in-time recovery - Multi-AZ deployment for high availability - Encryption at rest and in transit

AWS S3: - Model artifact storage - Training dataset backups - Log file archival - Version-controlled model storage

AWS Kinesis: - Data Streams: Real-time data ingestion - Data Firehose: Batch data delivery to S3/RDS - Data Analytics: Stream processing (if needed)

Monitoring and Logging

AWS CloudWatch: - Metrics: Model performance, latency, throughput - Logs: Application and system logs - Alarms: Automated alerting on thresholds - Dashboards: Real-time monitoring views

AWS X-Ray: - Distributed tracing for debugging - Performance bottleneck identification

Development Tools

Data Science Environment

Jupyter Notebooks: - `data_understanding.ipynb` - Exploratory data analysis - `data_preparation.ipynb` - Data preprocessing - `etl.ipynb` - ETL orchestration - `apt_modeling.ipynb` - APT model development - `itd_modeling.ipynb` - ITD model development - `phishing_modeling.ipynb` - Phishing model development - `zeroday_modeling.ipynb` - Zero-day model development - `*_modeling_evaluation.ipynb` - Model evaluation notebooks

Python Libraries: - `pandas`, `numpy` - Data manipulation - `xgboost` - Machine learning models - `scikit-learn` - Preprocessing and evaluation - `psycopg2`, `sqlalchemy` - Database connectivity - `matplotlib`, `seaborn` - Visualization - `imbalanced-learn` - Handling class imbalance

Database Tools

PostgresAgent.py: - Custom Python class for PostgreSQL operations - DDL/DML execution - DataFrame integration - Connection management

SQL Magic (Jupyter): - Direct SQL execution in notebooks - Seamless database integration

Environment Management

env_setup.py: - Environment variable loading (`.env` file) - Database connection configuration - SQL magic setup - PostgresAgent initialization

CI/CD Pipeline

Version Control: - Git repository for code management - Branching strategy: main, develop, feature branches

Continuous Integration: - Automated testing on code commits - Code quality checks (linting, formatting) - Model validation tests

Continuous Deployment: - Automated model deployment to staging - Integration testing in staging environment - Manual approval for production deployment - Rollback capabilities

Security Infrastructure

AWS IAM: - Role-based access control - Least privilege principles - Service-specific roles for Lambda, SageMaker

AWS Secrets Manager: - Database credentials - API keys - Encryption keys

VPC Configuration: - Isolated network environments - Security groups for access control - Private subnets for database access

Data Quality and Validation Framework

Data Quality Dimensions

Completeness

Target: >95% data completeness

Current State: 5-10% missing entries

Validation Rules: - Check for NULL values in critical fields (timestamp, IPs, ports) - Monitor missing data rates by client and time period - Alert when missing data exceeds 10% threshold

Remediation: - Imputation strategies for missing values - Statistical imputation (mean, median, mode) - Domain-specific imputation rules - Flag imputed values for model transparency

Accuracy

Validation Checks: - IP address format validation - Port number range validation (0-65535) - Protocol value validation (TCP, UDP, ICMP) - Timestamp consistency checks - Byte/packet count logical consistency

Data Quality Metrics: - Percentage of records passing validation - Error rate by validation rule - Client-specific accuracy scores

Consistency

Standardization Requirements: - Consistent time zone handling (UTC) - Standardized protocol naming - Uniform service classification - Consistent IP address formatting

Cross-Client Consistency: - Format standardization across client data - Unit of measurement consistency - Naming convention alignment

Timeliness

Real-time Processing: - Data ingestion latency < 5 seconds - Model inference latency < 100ms - End-to-end alert generation < 10 seconds

Batch Processing: - Daily batch jobs complete within 2 hours - Historical data processing SLA: 24 hours

Data Validation Procedures

Pre-Ingestion Validation

Schema Validation: - Verify column names and types - Check required fields present - Validate data ranges

Business Rule Validation: - Logical consistency checks (e.g., bytes >= packets) - Temporal consistency (timestamps in order) - Network topology validation (IP ranges)

Post-Processing Validation

Feature Engineering Validation: - Verify derived features calculated correctly - Check for NaN or infinite values - Validate feature distributions

Model Input Validation: - Ensure all required features present - Check feature value ranges - Validate data types match model expectations

Data Quality Monitoring

Automated Monitoring

CloudWatch Metrics: - Missing data percentage - Validation failure rate - Data freshness (time since last update) - Processing error counts

Alerting: - Email/SNS alerts for quality threshold breaches - Dashboard visualization of quality metrics - Weekly quality reports

Manual Quality Reviews

Weekly Reviews: - Review quality metrics dashboard - Investigate anomalies - Client-specific quality assessments

Monthly Audits: - Comprehensive data quality assessment - Trend analysis - Improvement recommendations

Data Cleansing Procedures

Automated Cleansing

Standardization: - Normalize IP addresses - Standardize protocol names - Convert time zones to UTC - Round numeric precision

Error Correction: - Fix common formatting errors - Correct obvious data entry mistakes - Remove duplicate records

Manual Intervention

Complex Cases: - Ambiguous data requiring domain expertise - Client-specific data issues - Systematic data quality problems

Documentation: - Log all data cleansing actions - Maintain audit trail - Document cleansing rules and rationale

Security and Privacy Controls

Data Protection Measures

Encryption

Data at Rest: - AWS RDS encryption enabled (AES-256) - S3 bucket encryption (SSE-S3 or SSE-KMS) - Model artifact encryption - Database backup encryption

Data in Transit: - TLS 1.2+ for all database connections - HTTPS for API communications - VPN for client data ingestion - Encrypted Kinesis streams

Access Controls

AWS IAM Policies: - Principle of least privilege - Role-based access control (RBAC) - Service-specific IAM roles - Regular access reviews and audits

Database Access: - PostgreSQL user authentication - Row-level security (if needed) - Connection IP whitelisting - Encrypted credentials in Secrets Manager

Network Security: - VPC isolation - Security groups restricting access - Private subnets for sensitive resources - Network ACLs for additional protection

Compliance Requirements

Data Privacy Regulations

GDPR Compliance: - Right to erasure (data deletion) - Data minimization principles - Privacy by design - Data processing agreements

Data Residency: - Client-specific data residency requirements - Regional data storage options - Cross-border data transfer restrictions

Industry Standards

SOC 2 Compliance: - Security controls documentation - Regular security audits - Access control procedures - Incident response procedures

ISO 27001 Alignment: - Information security management - Risk assessment procedures - Security control implementation

Privacy Safeguards

Data Anonymization

PII Protection: - IP address anonymization (if required) - Client data segregation - Pseudonymization techniques - Data masking for non-production environments

Data Usage Agreements

Client Consent: - Explicit consent for data usage - Clear data usage agreements - Purpose limitation - Data retention policies

Audit and Logging

Comprehensive Logging: - All data access logged - Model training data usage tracked - User activity audit trails - Compliance audit reports

Retention Policies: - Log retention: 90 days (minimum) - Data retention per client agreements - Secure log archival - Automated log purging

Ethical Considerations

Bias Mitigation

Model Fairness: - Bias testing across client segments - Fairness metrics monitoring - Regular bias audits - Mitigation strategies for identified biases

Transparency

Model Explainability: - Feature importance analysis - SHAP values for predictions - Decision boundary visualization - Client-facing explainability reports

Responsible AI

Ethical Review: - Ethics department consultation - Ethical use guidelines - Regular ethical reviews - Stakeholder feedback incorporation

Deployment Strategy and Environment Setup

Environment Architecture

Development Environment

Purpose: Model development and experimentation

Infrastructure: - Jupyter notebooks on local machines or SageMaker Notebooks - Development database (subset of production data) - Development S3 bucket for model artifacts - Limited compute resources

Access: - Data science team only - Read-only access to production data (anonymized subset) - Full access to development resources

Staging Environment

Purpose: Integration testing and validation

Infrastructure: - Production-like AWS environment - Full data pipeline replication - Model endpoints for testing - Monitoring and logging enabled

Access: - Extended team access (Data Science, Engineering, Security) - Production data access (with appropriate controls) - Full testing capabilities

Production Environment

Purpose: Live threat detection system

Infrastructure: - High-availability AWS setup - Multi-AZ deployment - Auto-scaling capabilities - Production-grade monitoring

Access: - Restricted access (Operations team) - Change management procedures - Approval workflows for deployments

Deployment Procedures

Model Deployment Workflow

Step 1: Model Validation - Performance metrics meet acceptance criteria - Security and compliance review completed - Documentation updated

Step 2: Staging Deployment - Deploy model to staging environment - Integration testing - Performance validation - Security testing

Step 3: Production Deployment - Create deployment package - Deploy to production (blue-green deployment) - Smoke tests - Monitor initial performance

Step 4: Verification - Verify model performance metrics - Check alert generation - Validate integration with SIEM/SOAR - Confirm monitoring dashboards operational

Containerization Strategy

Docker Images: - Base image: Python 3.9+ with required libraries - Model artifacts included in image - Environment-specific configuration - Health check endpoints

Kubernetes Deployment: - Deployment manifests for each model - Service definitions for load balancing - ConfigMaps for configuration - Secrets for credentials

Infrastructure as Code

AWS CloudFormation / Terraform: - Infrastructure definitions in version control - Reproducible environments - Environment-specific parameter files - Automated infrastructure provisioning

Rollback Procedures

Automated Rollback Triggers

Performance Degradation: - False positive rate exceeds threshold - Model accuracy drops below acceptable level - Latency exceeds SLA

System Failures: - Model endpoint failures - Database connection issues - Integration failures

Rollback Process

1. **Detection:** Automated monitoring detects issue
2. **Alert:** Operations team notified
3. **Decision:** Assess severity and decide on rollback
4. **Execution:** Revert to previous model version
5. **Verification:** Confirm system stability
6. **Investigation:** Root cause analysis

Rollback Capabilities

Model Versioning: - Previous model versions stored in S3 - Quick model endpoint updates - Database schema versioning (if needed)

Infrastructure Rollback: - Infrastructure changes in version control - Ability to revert infrastructure changes - Database backup restoration (if needed)

Infrastructure Provisioning

Automated Provisioning

CI/CD Pipeline: - Infrastructure provisioning on environment creation - Configuration management - Dependency resolution - Health checks

Manual Provisioning

Initial Setup: - VPC and networking setup - Database instance creation - S3 bucket creation - IAM role and policy setup - Monitoring and alerting configuration

Scaling: - Auto-scaling groups for compute - Database read replicas (if needed) - Kinesis shard scaling - S3 lifecycle policies

Monitoring and Maintenance Plan

Performance Tracking

Model Performance Metrics

Accuracy Metrics: - Precision, Recall, F1-Score per attack type - Overall accuracy - ROC-AUC scores - Confusion matrices

Operational Metrics: - Mean Time to Detect (MTTD) - False Positive Rate (target: <2%) - Alert generation rate - Model inference latency

Tracking Frequency: - Real-time: Key metrics updated every minute - Daily: Comprehensive performance reports - Weekly: Trend analysis and reporting

System Performance Metrics

Infrastructure Metrics: - CPU utilization - Memory usage - Network throughput - Database connection pool usage

Application Metrics: - Request rates - Error rates - Response times - Queue depths

Monitoring Dashboards

CloudWatch Dashboards: - Real-time model performance - System health overview - Alert trends - Resource utilization

Custom Dashboards: - Business KPI tracking - Client-specific metrics - Historical trend analysis

Model Drift Detection

Data Drift Monitoring

Feature Distribution Monitoring: - Statistical tests (KS test, PSI) comparing training vs. production data - Feature distribution visualizations - Automated alerts on significant drift

Concept Drift Detection: - Model performance degradation over time - Prediction distribution changes - Label distribution shifts (if available)

Detection Mechanisms

Automated Detection: - Daily statistical tests on feature distributions - Performance metric trend analysis - Anomaly detection on prediction patterns

Alerting: - Email/SNS alerts when drift detected - Severity-based alerting (warning, critical)
- Dashboard indicators

Retraining Schedules

Scheduled Retraining

Frequency: - Monthly retraining for all models - Weekly retraining if performance degrades - Ad-hoc retraining after major data changes

Retraining Triggers: - Model performance below threshold - Significant data drift detected - New attack patterns identified - Sufficient new labeled data available

Retraining Process

1. **Data Collection:** Gather new training data
2. **Data Validation:** Quality checks on new data
3. **Model Training:** Train new model version
4. **Validation:** Evaluate on holdout set
5. **Comparison:** Compare with current production model
6. **Deployment:** Deploy if performance improved
7. **Monitoring:** Track new model performance

Model Versioning

Version Management: - Semantic versioning (major.minor.patch) - Version metadata (training date, data snapshot, performance) - Version comparison and rollback capabilities

Ongoing Maintenance Procedures

Regular Maintenance Tasks

Daily: - Review monitoring dashboards - Check for alerts and errors - Verify data pipeline health

Weekly: - Performance metric review - Data quality assessment - Client feedback review

Monthly: - Comprehensive system health review - Model performance analysis - Infrastructure optimization review - Security audit review

Proactive Maintenance

Capacity Planning: - Monitor resource utilization trends - Plan for scaling needs - Optimize resource allocation

Performance Optimization: - Database query optimization - Model inference optimization - Pipeline efficiency improvements

Security Updates: - Dependency vulnerability scanning - Security patch application - Access review and cleanup

Maintenance Windows

Scheduled Maintenance: - Monthly maintenance windows (low-traffic periods) - Advance notification to stakeholders - Minimal service disruption

Emergency Maintenance: - Critical security patches - System stability issues - Immediate remediation required

Testing and Validation Approach

Unit Testing

Model Component Testing

Feature Engineering Tests: - Test feature calculation functions - Validate encoding procedures - Verify feature transformations - Test edge cases and missing values

Data Processing Tests: - Database query validation - Data transformation correctness - Data quality check functions - Error handling

Code Quality

Test Coverage: - Target: >80% code coverage - Critical functions: 100% coverage - Regular coverage reports

Testing Framework: - `pytest` for Python unit tests - Mock objects for external dependencies - Fixtures for test data

Integration Testing

Pipeline Integration Tests

ETL Pipeline Testing: - End-to-end data flow validation - Database integration tests - Error handling and recovery - Performance under load

Model Integration Tests: - Model loading and initialization - Feature extraction integration - Prediction generation - Alert generation workflow

System Integration Tests

AWS Service Integration: - Lambda function integration - SageMaker endpoint integration - Kinesis stream processing - CloudWatch logging and metrics

External System Integration: - SIEM integration testing - SOAR platform integration - Client portal integration - API endpoint testing

Model Validation Methods

Cross-Validation

Stratified K-Fold: - 5-fold cross-validation - Maintains class distribution - Robust performance estimates - Prevents overfitting

Holdout Validation

Train/Validation/Test Split: - 60% training, 20% validation, 20% test - Stratified splits - Temporal considerations (if applicable)

Temporal Validation

Time-Based Splits: - Train on historical data - Validate on recent data - Test on future data (if available) - Prevents temporal leakage

Performance Validation

Metrics Validation: - Precision, Recall, F1-Score - ROC-AUC, PR-AUC - Confusion matrix analysis - Per-class performance

Business Metric Validation: - MTTD reduction validation - False positive rate validation - Alert quality assessment

Acceptance Criteria

Model Performance Criteria

Minimum Performance Thresholds: - Precision: >85% per attack type - Recall: >80% per attack type - F1-Score: >82% per attack type - False Positive Rate: <2% - ROC-AUC: >0.90

Operational Criteria

System Performance: - Model inference latency: <100ms (p95) - End-to-end alert generation: <10 seconds - System uptime: >99.9% - Data pipeline latency: <5 seconds

Business Criteria

Business Impact: - MTTD reduction: 50% per attack type - Client satisfaction: Positive feedback - Integration success: Seamless SIEM/SOAR integration - Documentation completeness: All documentation updated

Test Data Management

Test Data Strategy

Synthetic Test Data: - Generate synthetic network flows - Known attack patterns - Edge cases and anomalies

Anonymized Production Data: - Anonymized subset of production data - Representative of real-world scenarios - Updated regularly

Labeled Test Sets: - Curated test sets with known labels - Balanced class distribution - Diverse attack scenarios

Test Environment

Isolated Test Environment: - Separate from development/staging/production - Controlled test data - Reproducible test scenarios

API and Interface Specifications

Model Inference API

REST API Endpoints

Endpoint: /api/v1/predict

Method: POST

Authentication: API key or OAuth 2.0

Content-Type: application/json

Request Body:

```
{
  "model_type": "apt" | "itd" | "phishing" | "zeroday",
  "features": {
    "timestamp": "2025-05-23T03:03:58Z",
    "source_ip": "222.187.194.161",
    "dest_ip": "172.16.173.30",
    "source_port": 50397,
    "dest_port": 25,
    "protocol": "TCP",
    "duration": 17.28,
    "packets": 53,
    "bytes": 37361,
    ...
  }
}
```

Response:

```
{
  "prediction": true,
  "probability": 0.87,
  "confidence": "high",
  "model_version": "1.2.3",
}
```

```
"timestamp": "2025-05-23T03:03:59Z"  
}
```

Error Responses: - 400: Bad Request (invalid input) - 401: Unauthorized - 500: Internal Server Error

Batch Prediction API

Endpoint: /api/v1/batch-predict

Method: POST

Request: Array of feature objects

Response: Array of predictions

SIEM/SOAR Integration

Splunk Integration

Splunk Add-on: - Custom add-on for CyberGuard models - Real-time alert ingestion - Dashboard integration

Data Format: - CEF (Common Event Format) - Structured data with threat scores - Metadata for correlation

QRadar Integration

QRadar DSM: - Custom Device Support Module - Log source configuration - Event parsing and normalization

SOAR Platform Integration

API Integration: - RESTful API for alert ingestion - Webhook support for real-time alerts - Playbook trigger integration

Alert Format:

```
{
  "alert_id": "uuid",
  "timestamp": "ISO 8601",
  "attack_type": "apt" | "itd" | "phishing" | "zeroday",
  "severity": "low" | "medium" | "high" | "critical",
  "source_ip": "string",
  "dest_ip": "string",
  "confidence": 0.0-1.0,
  "details": {
    "model_version": "string",
    "features": {}
  }
}
```

Client Portal Interface

Dashboard API

Endpoint: /api/v1/dashboard/metrics

Method: GET

Query Parameters: - start_date: ISO 8601 date - end_date: ISO 8601 date - attack_type: Optional filter

Response:

```
{
  "metrics": {
    "total_alerts": 1234,
    "false_positives": 25,
    "mttd_reduction": 0.52,
    "attack_breakdown": {
      "apt": 450,
      "itd": 320,
      "phishing": 380,
      "zeroday": 84
    }
  },
  "time_series": [...]
}
```

Alert Management API

Endpoint: `/api/v1/alerts`

Methods: - GET: List alerts (with filtering, pagination) - GET `/api/v1/alerts/{id}`: Get alert details - PUT `/api/v1/alerts/{id}`: Update alert status - POST `/api/v1/alerts/{id}/feedback`: Provide feedback

Data Formats

Input Data Format

Network Flow JSON: - Standardized JSON schema - Required vs. optional fields - Data type specifications - Validation rules

Output Data Format

Prediction Response: - Consistent response structure - Error message format - Metadata inclusion

Communication Protocols

HTTP/HTTPS

Protocol: HTTPS (TLS 1.2+)

Methods: GET, POST, PUT, DELETE

Status Codes: Standard HTTP status codes

Message Queue (Optional)

AWS SQS: - Asynchronous processing - High-volume scenarios - Decoupled architecture

WebSocket (Real-time)

WebSocket API: - Real-time alert streaming - Client dashboard updates - Low-latency requirements

Scalability and Performance Considerations

Expected Load

Current Estimates

Data Volume: - ~5 million network flow records per dataset - Continuous real-time ingestion
- Growing data volumes over time

Request Rates: - Real-time predictions: 1,000-10,000 requests/second (peak) - Batch processing: Millions of records per day - Dashboard queries: 100-1,000 requests/minute

Growth Projections

3-Year Projections: - 3x increase in data volume - 5x increase in prediction requests - 10+ new clients - Additional attack types

Scaling Strategies

Horizontal Scaling

Compute Scaling: - Auto-scaling groups for Lambda functions - Kubernetes horizontal pod autoscaling - SageMaker endpoint auto-scaling - Load balancing across instances

Database Scaling: - Read replicas for query distribution - Connection pooling - Query optimization - Partitioning strategies (if needed)

Vertical Scaling

Resource Optimization: - Right-size instances based on usage - Memory optimization for models - CPU optimization for processing - Storage optimization

Caching Strategies

Model Caching: - In-memory model caching - Feature caching for repeated predictions - Result caching for similar queries

Data Caching: - Frequently accessed data in cache - Redis/ElastiCache for caching - Cache invalidation strategies

Performance Benchmarks

Latency Targets

Model Inference: - P50 latency: <50ms - P95 latency: <100ms - P99 latency: <200ms

End-to-End Processing: - Data ingestion to alert: <10 seconds - Batch processing: 1M records in <1 hour

Throughput Targets

Real-time Processing: - 10,000 predictions/second (per model) - 100,000 network flows/second ingestion

Batch Processing: - 1 million records/hour processing capacity

Resource Optimization

Cost Optimization

Right-Sizing: - Monitor and adjust instance sizes - Reserved instances for predictable workloads - Spot instances for batch processing - S3 lifecycle policies for data archival

Efficiency Improvements: - Model quantization (if applicable) - Batch processing optimization - Database query optimization - Pipeline efficiency improvements

Performance Tuning

Database Optimization: - Index optimization - Query plan analysis - Connection pool tuning - Vacuum and analyze schedules

Model Optimization: - Model compression techniques - Inference optimization - Batch prediction optimization - GPU utilization (if available)

Capacity Planning

Monitoring and Alerts

Capacity Metrics: - Resource utilization trends - Growth rate projections - Capacity threshold alerts - Scaling trigger points

Planning Process

Quarterly Reviews: - Review capacity metrics - Project future needs - Plan infrastructure changes - Budget planning

Risk Mitigation and Contingency Plans

Technical Risks

Model Performance Degradation

Risk: Models fail to meet performance targets or degrade over time

Probability: Medium

Impact: High

Mitigation: - Comprehensive model validation - Continuous performance monitoring - Automated retraining triggers - Model versioning and rollback

Contingency: - Immediate rollback to previous model version - Emergency retraining with latest data - Manual review and adjustment

Data Quality Issues

Risk: Poor data quality impacts model performance

Probability: Medium

Impact: High

Mitigation: - Robust data validation framework - Automated quality monitoring - Data quality alerts - Regular quality audits

Contingency: - Pause model training until data quality restored - Data cleansing procedures - Client communication and remediation

Infrastructure Failures

Risk: AWS service outages or infrastructure failures

Probability: Low

Impact: Critical

Mitigation: - Multi-AZ deployment - Automated failover - Regular backup procedures - Disaster recovery testing

Contingency: - Failover to backup region (if configured) - Manual intervention procedures - Client communication protocols - Service restoration procedures

Integration Failures

Risk: SIEM/SOAR integration failures

Probability: Medium

Impact: Medium

Mitigation: - Comprehensive integration testing - API versioning - Fallback mechanisms - Monitoring and alerting

Contingency: - Alternative integration methods - Manual alert distribution - Temporary workarounds - Integration team escalation

Data and Security Risks

Data Breach

Risk: Unauthorized access to sensitive data

Probability: Low

Impact: Critical

Mitigation: - Comprehensive security controls - Encryption at rest and in transit - Access controls and monitoring - Regular security audits

Contingency: - Incident response procedures - Data breach notification protocols - Forensic investigation - Legal and compliance coordination

Compliance Violations

Risk: Failure to meet regulatory requirements

Probability: Low

Impact: High

Mitigation: - Regular compliance reviews - Legal and ethics consultation - Compliance monitoring - Documentation and audit trails

Contingency: - Immediate compliance remediation - Legal consultation - Regulatory notification (if required) - Process improvement

Business Risks

Project Delays

Risk: Timeline delays impact business objectives

Probability: Medium

Impact: Medium

Mitigation: - Realistic timeline planning - Regular progress monitoring - Risk identification and mitigation - Contingency time buffers

Contingency: - Timeline adjustment - Resource reallocation - Scope prioritization - Stakeholder communication

Budget Overruns

Risk: Project costs exceed budget

Probability: Medium

Impact: Medium

Mitigation: - Detailed budget planning - Regular cost monitoring - Cost optimization efforts - Contingency fund allocation

Contingency: - Budget review and approval - Cost reduction measures - Scope adjustment - Executive escalation

Failure Scenarios

Complete System Failure

Scenario: Complete system outage

Response: 1. Immediate incident response activation 2. System status assessment 3. Failover to backup systems (if available) 4. Manual alerting procedures 5. Client communication 6. Root cause analysis 7. System restoration 8. Post-incident review

Model Failure

Scenario: All models fail simultaneously

Response: 1. Detect failure via monitoring 2. Alert operations team 3. Investigate root cause 4. Rollback to previous model versions 5. Emergency retraining (if needed) 6. Client notification 7. Performance validation

Data Pipeline Failure

Scenario: Data ingestion or processing pipeline fails

Response: 1. Detect pipeline failure 2. Assess data loss/gap 3. Pause model training (if needed) 4. Pipeline restoration 5. Data backfill procedures 6. Validation and verification 7. Resume normal operations

Risk Monitoring

Risk Register

Maintenance: - Regular risk assessment updates - Risk probability and impact reviews - Mitigation effectiveness evaluation - New risk identification

Risk Reporting

Frequency: - Weekly risk status updates - Monthly comprehensive risk review - Quarterly risk assessment - Ad-hoc risk escalation

Pilot Phase Implementation Plan

Pilot Scope

Target User Groups

Primary Users: - 2-3 select enterprise clients (volunteer participants) - Client SOC teams (5-10 analysts per client) - CyberGuard operations team

Selection Criteria: - Clients with robust security infrastructure - Willingness to provide feedback - Representative network environments - Strong relationship with CyberGuard

Limited Feature Sets

Pilot Features: - 2 attack types: APT and Phishing (highest priority) - Real-time detection only (batch processing excluded) - Basic alerting (advanced analytics excluded) - Single SIEM integration (Splunk or QRadar)

Excluded Features: - ITD and Zero-day models (post-pilot) - Advanced analytics and reporting - Multiple SIEM integrations - Client portal (basic dashboard only)

Data Subsets

Pilot Data: - 6 months of historical data per client - Real-time data from pilot clients only - Anonymized data for testing - Limited to 2 attack types

Data Volume: - ~1 million records per client - Real-time: 1,000-5,000 flows/second - Manageable for initial testing

Pilot Timeline

Phase 1: Preparation (2 weeks)

Week 1: - Client selection and agreements - Pilot environment setup - Data access and integration setup - Team training

Week 2: - Model deployment to pilot environment - Integration testing - Monitoring setup - Documentation preparation

Phase 2: Soft Launch (1 week)

Activities: - Deploy to pilot clients (read-only mode) - Shadow monitoring (no active alerting) - System validation - Performance baseline establishment

Phase 3: Active Pilot (4 weeks)

Week 1-2: - Enable active alerting - Monitor performance and feedback - Daily check-ins with client teams - Issue identification and resolution

Week 3-4: - Refinement based on feedback - Performance optimization - Additional training if needed - Data collection for evaluation

Phase 4: Evaluation (1 week)

Activities: - Comprehensive performance evaluation - Client feedback collection - Success criteria assessment - Go/no-go decision preparation

Total Duration: 8 weeks

Success Metrics

Technical Metrics

Model Performance: - Precision: >80% (pilot target, <85% production) - Recall: >75% (pilot target, <80% production) - False Positive Rate: <3% (pilot target, <2% production) - MTTD reduction: 40% (pilot target, 50% production)

System Performance: - Uptime: >99% - Latency: <200ms (pilot target, <100ms production) - Integration stability: <5 integration errors/week

Business Metrics

User Adoption: - Active users: >80% of target SOC analysts - Daily usage: >4 hours per analyst - Positive feedback: >70% satisfaction rate

Operational Metrics: - Alert accuracy: >75% actionable alerts - Time savings: 30% reduction in manual analysis - Client retention: No pilot client churn

Rollback Procedures

Automatic Rollback Triggers

Performance Thresholds: - False positive rate >5% - System uptime <95% - Critical integration failures - Data quality issues

Manual Rollback Process

1. **Decision:** Pilot team decides on rollback
2. **Notification:** Notify stakeholders
3. **Execution:** Disable alerting, revert to previous state
4. **Investigation:** Root cause analysis
5. **Remediation:** Fix issues
6. **Re-deployment:** Resume pilot after fixes

Rollback Criteria

Immediate Rollback: - Security incidents - Data breaches - Critical system failures - Compliance violations

Planned Rollback: - Performance below acceptable thresholds - Client requests - Major integration issues - Extended system instability

Criteria for Advancing to Production

Technical Criteria

Must Have: - All technical success metrics met - System stability demonstrated (4 weeks) - Integration reliability confirmed - Performance targets achieved

Should Have: - Model performance within 5% of production targets - Zero critical security issues - Comprehensive documentation complete - Monitoring and alerting validated

Business Criteria

Must Have: - Positive client feedback (>70% satisfaction) - Client commitment to production deployment - Business value demonstrated - ROI projections validated

Should Have: - Additional client interest - Positive stakeholder feedback - Budget approval for production - Resource availability confirmed

Go/No-Go Decision

Decision Makers: - Executive sponsor (Eleanor Vance) - Technical lead - Client representatives - Project manager

Decision Timeline: - Evaluation week: Comprehensive assessment - Decision meeting: Week 9 - Production planning: If go decision, immediate planning

Pilot Phase Business Value Estimation

Baseline Measurements

Current State Metrics

Detection Performance: - Current MTTR: 4-6 hours (varies by attack type) - Current False Positive Rate: 8-12% - Current Precision: 65-75% - Current Recall: 60-70%

Operational Metrics: - Manual analysis time: 2-3 hours per alert - SOC analyst capacity: 50-100 alerts/day - Client satisfaction: 75% (baseline) - Alert fatigue: High (frequent false positives)

Financial Baseline: - Current detection costs: \$X per incident - Client retention rate: 85% - Revenue per client: \$Y annually - Operational efficiency: Baseline established

Projected Improvements

Detection Performance Improvements

MTTD Reduction: - Target: 50% reduction (2-3 hours) - Pilot target: 40% reduction (2.4-3.6 hours) - Estimated improvement: 1.6-2.4 hours saved per detection

False Positive Reduction: - Target: <2% (from 8-12%) - Pilot target: <3% (from 8-12%) - Estimated: 70-80% reduction in false positives

Accuracy Improvements: - Precision: 65-75% → 80-85% (pilot: 75-80%) - Recall: 60-70% → 75-80% (pilot: 70-75%) - Overall accuracy improvement: 15-20%

Operational Efficiency Gains

Time Savings: - Reduced manual analysis: 30-40% time savings - Faster threat response: 40-50% faster - Reduced alert fatigue: 70-80% fewer false positives - Estimated: 1-1.5 hours saved per analyst per day

Capacity Improvements: - Increased analyst capacity: 30-40% more alerts handled - Better resource utilization - Improved analyst satisfaction

Business Impact

Client Retention: - Current: 85% retention rate - Target: 90-95% retention (5-10% improvement) - Pilot: Maintain 85%+ (no degradation)

Client Acquisition: - Enhanced value proposition - Competitive differentiation - Market positioning improvement

Risk-Adjusted Value Calculations

Revenue Impact

Client Retention Value: - Current clients: N clients - Retention improvement: 5-10% - Revenue per client: \$Y annually - 3-year retention value: \$Z (calculated)

New Client Acquisition: - Target: 10-15% increase in 2 years - Average client value: \$Y annually - 3-year acquisition value: \$W (calculated)

Service Expansion: - Enhanced service offerings - Premium pricing potential - Additional revenue streams

Cost Savings

Operational Efficiency: - Reduced manual analysis time - Lower false positive handling costs - Improved resource utilization - Estimated annual savings: \$X

Incident Cost Reduction: - Faster detection = lower incident costs - Reduced data exfiltration - Lower remediation costs - Estimated savings per incident: \$Y

Pilot Phase ROI

Pilot Investment: - Infrastructure: \$X - Personnel: \$Y - Client engagement: \$Z - Total pilot cost: \$Total

Pilot Returns: - Efficiency gains: \$A - Client satisfaction: Qualitative - Learning and refinement: Value in production success - Risk mitigation: Value of identifying issues early

Pilot ROI Calculation: - Net pilot value: Returns - Investment - ROI: $(\text{Net Value} / \text{Investment}) \times 100\%$ - Break-even analysis

Methodology for Measuring Outcomes

Measurement Framework

Data Collection: - Automated metrics collection - Client feedback surveys - Operational logs and reports - Financial tracking

Measurement Frequency: - Daily: Automated metrics - Weekly: Performance reports - Monthly: Comprehensive analysis - End of pilot: Final evaluation

Key Performance Indicators

Technical KPIs: - MTTD reduction percentage - False positive rate - Precision and recall - System uptime and latency

Operational KPIs: - Time savings per analyst - Alert handling capacity - Analyst satisfaction scores - Integration stability

Business KPIs: - Client satisfaction scores - Client retention rate - Cost savings realized - Revenue impact (if measurable in pilot)

Comparison Methodology

Before/After Comparison: - Baseline metrics established pre-pilot - Pilot metrics collected during pilot - Comparison and delta calculation - Statistical significance testing

Control Group (if applicable): - Compare pilot clients vs. non-pilot clients - Isolate pilot impact - Validate improvements

Reporting

Weekly Reports: - Performance metrics dashboard - Issue tracking and resolution - Client feedback summary

Monthly Reports: - Comprehensive performance analysis - Business value assessment - Risk and issue management

Final Pilot Report: - Complete performance evaluation - Business value quantification - Lessons learned - Production readiness assessment - Go/no-go recommendation

Production Phase Implementation Plan

Phased Rollout Strategy

Phase 1: Foundation (Weeks 1-4)

Infrastructure Scaling: - Production AWS environment setup - Database scaling and optimization - High-availability configuration - Monitoring and alerting expansion

Model Deployment: - Deploy all 4 models to production - Model endpoint configuration - Load testing and optimization - Performance validation

Integration Setup: - SIEM/SOAR integrations for all clients - API endpoint deployment - Client portal deployment - Integration testing

Phase 2: Limited Production (Weeks 5-8)

Client Onboarding: - Onboard 5-10 priority clients - Gradual traffic increase - Close monitoring and support - Issue resolution and optimization

Feature Activation: - Enable all 4 attack type models - Activate batch processing - Enable advanced analytics - Client portal features

Phase 3: Full Production (Weeks 9-12)

Scale-Up: - Onboard remaining clients - Full traffic volume - All features active - Production-grade operations

Optimization: - Performance tuning - Cost optimization - Process refinement - Documentation completion

User Onboarding Procedures

Client Onboarding Process

Pre-Onboarding: - Client agreement and contracts - Data access setup - Security and compliance review - Integration requirements assessment

Onboarding Steps: 1. **Kickoff Meeting:** Project introduction, timeline, expectations 2. **Technical Setup:** SIEM/SOAR integration, API access, data pipeline 3. **Training Delivery:** SOC team training, documentation, best practices 4. **Soft Launch:** Shadow mode (1 week), monitoring only 5. **Go-Live:** Active alerting, full functionality 6. **Support Period:** 2-4 weeks intensive support

Training Delivery

SOC Team Training: - Model capabilities and limitations - Alert interpretation and prioritization - Integration with existing workflows - Best practices and troubleshooting

Training Format: - Live training sessions (4-6 hours) - Video tutorials and documentation - Hands-on exercises - Q&A sessions

Training Timeline: - Week 1: Training delivery - Week 2: Practice and support - Week 3: Go-live preparation - Ongoing: Refresher training as needed

Change Management Protocols

Communication Strategy

Stakeholder Communication: - Executive updates (monthly) - Client communications (weekly during onboarding) - Team updates (weekly) - Status reports and dashboards

Change Announcements: - Advance notice for major changes - Change impact assessment - Rollback plans communicated - Support availability

Change Control Process

Change Types: - **Standard:** Routine updates, low risk - **Normal:** Moderate changes, some risk - **Emergency:** Critical fixes, immediate need

Change Process: 1. Change request submission 2. Impact assessment 3. Approval (based on change type) 4. Implementation 5. Verification 6. Documentation

Rollback Procedures

Change Rollback: - Automated rollback for failed deployments - Manual rollback procedures - Communication protocols - Post-rollback analysis

System Cutover Activities

Cutover Planning

Cutover Strategy: - Blue-green deployment - Gradual traffic migration - Zero-downtime cutover - Rollback capabilities

Cutover Timeline: - Pre-cutover: Final testing and validation - Cutover window: Low-traffic period - Post-cutover: Monitoring and validation - Stabilization: 24-48 hours

Cutover Checklist

Pre-Cutover: - ☐ All testing completed - ☐ Stakeholders notified - ☐ Rollback plan prepared - ☐ Support team on standby - ☐ Monitoring dashboards ready

Cutover: - ☐ Traffic migration initiated - ☐ System health monitoring - ☐ Performance validation - ☐ Issue detection and resolution

Post-Cutover: - ☐ System stability confirmed - ☐ Performance metrics validated - ☐ Stakeholder notification - ☐ Documentation updated

Production Infrastructure Scaling

Infrastructure Requirements

Compute Resources: - Auto-scaling groups configured - Load balancers deployed - Sufficient capacity for peak loads - Multi-AZ deployment

Database Scaling: - Production database instance sizing - Read replicas for query distribution - Backup and recovery configured - Performance monitoring

Storage: - S3 buckets for model artifacts - Sufficient storage capacity - Lifecycle policies configured - Backup and archival

Scaling Triggers

Automatic Scaling: - CPU utilization >70% - Memory usage >80% - Request queue depth >threshold - Latency >target

Manual Scaling: - Planned capacity increases - Seasonal adjustments - Growth projections

Data Migration Procedures

Migration Strategy

Data Migration: - Historical data migration (if needed) - Incremental data sync - Data validation and verification - Migration rollback plan

Migration Timeline: - Pre-migration: Planning and preparation - Migration: Execution and monitoring - Post-migration: Validation and cleanup

Migration Validation

Data Validation: - Record count verification - Data integrity checks - Performance validation - Client verification

User Access Provisioning

Access Management

User Provisioning: - Client user account creation - Role-based access assignment - API key generation - Initial password setup

Access Controls: - Least privilege principles - Client data isolation - Audit logging - Regular access reviews

Access Types

Portal Access: - Dashboard access - Alert management - Reporting access

API Access: - API key authentication - Rate limiting - Usage monitoring

Integration Access: - SIEM/SOAR integration credentials - Secure connection setup - Access monitoring

Training Delivery Timelines

Training Schedule

Week 1: Initial Training - Day 1-2: System overview and capabilities - Day 3-4: Hands-on training - Day 5: Q&A and assessment

Week 2: Advanced Training - Day 1-2: Advanced features and best practices - Day 3-4: Troubleshooting and support - Day 5: Certification (if applicable)

Ongoing: - Monthly refresher sessions - New feature training - Best practice updates

Communication Strategies

Internal Communication

Team Communication: - Daily standups - Weekly team meetings - Slack/Teams channels - Documentation wiki

Client Communication

Client Updates: - Weekly status reports - Monthly performance reviews - Quarterly business reviews - Ad-hoc communications

Support Channels: - Help desk/ticketing system - Email support - Phone support (for critical issues) - Client portal messaging

Organization-Wide Adoption

Change Management: - Executive sponsorship - Champion network - Success stories and case studies - Training and support

Adoption Metrics: - User adoption rates - Feature utilization - Client satisfaction - Business impact

Post-Production Monitoring Plan

Automated Monitoring Dashboards

Real-Time Dashboards

System Health Dashboard: - Infrastructure metrics (CPU, memory, network) - Service health status - Error rates and types - Latency metrics

Model Performance Dashboard: - Prediction accuracy metrics - False positive rates - MTTD trends - Per-attack-type performance

Business Metrics Dashboard: - Alert volumes and trends - Client usage statistics - Business KPI tracking - Revenue impact metrics

Custom Dashboards

Client-Specific Dashboards: - Per-client performance metrics - Client alert trends - Client satisfaction indicators

Operational Dashboards: - Team performance metrics - Support ticket trends - System utilization - Cost tracking

Alert Systems

Alert Configuration

Alert Types: - **Critical:** System failures, security incidents - **Warning:** Performance degradation, capacity issues - **Info:** Status updates, scheduled events

Alert Channels: - Email notifications - SMS (for critical alerts) - Slack/Teams integration - PagerDuty (for on-call)

Alert Thresholds

Performance Alerts: - False positive rate $>2.5\%$ - Model accuracy $<80\%$ - MTTD increase $>20\%$ - System latency $>200\text{ms}$ (p95)

System Alerts: - System uptime $<99\%$ - Error rate $>1\%$ - Database connection failures - Integration failures

Business Alerts: - Client churn risk - Unusual alert volume changes - Revenue impact indicators

Performance Tracking Mechanisms

Model Performance Monitoring

Continuous Monitoring: - Real-time prediction accuracy - Per-attack-type performance - Feature drift detection - Model version comparison

Performance Metrics: - Precision, Recall, F1-Score (daily) - ROC-AUC (weekly) - Confusion matrices (weekly) - MTTD trends (daily)

System Performance Monitoring

Infrastructure Metrics: - Resource utilization trends - Capacity planning indicators - Cost optimization opportunities - Performance bottlenecks

Application Metrics: - Request rates and patterns - Error rates and types - Latency distributions - Throughput metrics

Model Performance Degradation Detection

Detection Mechanisms

Automated Detection: - Statistical tests on performance metrics - Trend analysis and anomaly detection - Comparison with baseline performance - Alert generation on degradation

Detection Frequency: - Real-time: Continuous monitoring - Daily: Comprehensive performance review - Weekly: Trend analysis - Monthly: Detailed performance audit

Degradation Response

Response Procedures: 1. **Detection:** Automated or manual detection 2. **Investigation:** Root cause analysis 3. **Assessment:** Severity and impact evaluation 4. **Action:** Retraining, rollback, or adjustment 5. **Validation:** Performance verification 6. **Documentation:** Incident logging

Data Quality Monitoring

Quality Metrics

Completeness Monitoring: - Missing data percentages - Data freshness indicators - Gap detection - Client-specific quality metrics

Accuracy Monitoring: - Validation failure rates - Data error detection - Consistency checks - Client data quality scores

Quality Alerts

Alert Thresholds: - Missing data >10% - Validation failures >5% - Data freshness >1 hour delay - Quality score <90%

Response Procedures: - Immediate investigation - Client notification (if needed) - Data quality remediation - Root cause analysis

Business Impact Measurement

Business Metrics

Client Metrics: - Client retention rates - Client satisfaction scores - Client usage statistics - Client value metrics

Revenue Metrics: - Revenue per client - Revenue growth trends - Service expansion metrics - ROI tracking

Operational Metrics: - Cost per detection - Efficiency gains - Resource utilization - Operational savings

Impact Reporting

Reporting Frequency: - Weekly: Operational metrics - Monthly: Business impact report - Quarterly: Comprehensive business review - Annually: ROI assessment

Report Contents: - Performance summary - Business impact analysis - Client feedback - Recommendations - Action items

Ongoing Maintenance Procedures

Regular Maintenance

Daily: - Dashboard review - Alert monitoring - Issue resolution - Performance checks

Weekly: - Performance analysis - Quality assessment - Client feedback review - Team coordination

Monthly: - Comprehensive system review - Performance optimization - Capacity planning - Security audit

Quarterly: - Business review - Strategic planning - Technology assessment - Roadmap updates

Proactive Maintenance

Continuous Improvement: - Performance optimization - Cost optimization - Feature enhancements - Process improvements

Technology Updates: - Dependency updates - Security patches - Infrastructure improvements - Tool and technology evaluation

Maintenance Schedule

Scheduled Maintenance Windows

Monthly Maintenance: - Low-traffic period (e.g., Sunday 2-4 AM) - Advance notification (1 week) - Minimal service disruption - Rollback plan

Quarterly Maintenance: - Comprehensive system updates - Extended maintenance window - Client communication - Post-maintenance validation

Emergency Maintenance

Procedures: - Immediate response team activation - Impact assessment - Client notification - Rapid resolution - Post-incident review

Appendices

Appendix A: Glossary

APT: Advanced Persistent Threat

ITD: Insider Threat Detection

MTTD: Mean Time to Detect

SIEM: Security Information and Event Management

SOAR: Security Orchestration, Automation and Response

ROC-AUC: Receiver Operating Characteristic - Area Under Curve

PSI: Population Stability Index

KS Test: Kolmogorov-Smirnov Test

Appendix B: References

- Business Understanding Document
- CRISP-DM Session Transcript
- AWS Documentation
- XGBoost Documentation
- Industry Best Practices

Appendix C: Document Version History

Version	Date	Author	Changes
1.0	2025-11-XX	Data Science Team	Initial design document

Document Status: Draft

Next Review Date: TBD

Approval Required: Executive Sponsor, Technical Lead, Security Specialist